arXiv:2602.21477v1 [cs.MA] 25 Feb 2026

Pancake: Hierarchical Memory System for Multi-Agent LLM Serving

Zhengding Hu, Zaifeng Pan, Prabhleen Kaur, Vibha Murthy, Zhongkai Yu, Yue Guan,
Zhen Wang, Steven Swanson, Yufei Ding
Computer Science and Engineering, University of California, San Diego

Abstract

In this work, we identify and address the core challenges of agentic memory management in LLM serving, where large-scale storage, frequent updates, and multiple coexisting agents jointly introduce complex and high-cost approximate nearest neighbor (ANN) searching problems. We present Pancake, a multi-tier agentic memory system that unifies three key techniques: (i) multi-level index caching for single agents, (ii) coordinated index management across multiple agents, and (iii) collaborative GPU–CPU acceleration. Pancake exposes easy-to-use interface that can be integrated into memory-based agents like Mem-GPT, and is compatible with agentic frameworks such as LangChain and LlamaIndex. Experiments on realistic agent workloads show that Pancake substantially outperforms existing frameworks, achieving more than 4.29× end-to-end throughput improvement.

1 Introduction

Agents have emerged as one of the defining paradigms of the LLM era, enabling complex task scenarios including task planning [25,58], knowledge organization [5,18], tool-augmented generation [57,76] and even scientific researches [47,64]. These complex tasks, often carried out through multi-turn environment interaction [77], self-reflection [59], or multi-agent collaboration [39], has introduced substantial information into the generation process and poses significant challenges for context length and attention fidelity [45].

In response to this trend, **Agent Memory** [82] has emerged as a key mechanism to manage complex contexts and enhance generation quality. Unlike prior Retrieval-Augmented Generation (RAG) methods [5,29,30,37,38,71], which rely on a static knowledge database and lack the ability to capture an agent's runtime state, results, and other dynamic information, agent memory maintains an external database that records essential information, including external knowledge [30,53], action history [73,88], user profile [21,67], and more. The agent must retrieve the most relevant memory items to guide generation throughout LLM inference, while also dynamically inserting newly generated items for future reference.

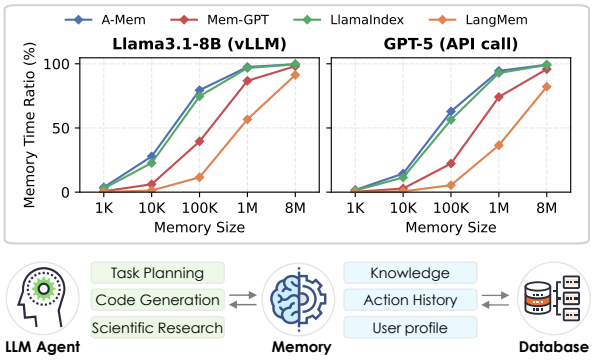

Figure 1: Memory-based workflow of agentic LLMs.

Agent Memory enables reliable, consistent, and progressively refined agent outputs over complex tasks. Yet, such a continuous memory mechanism also introduces a highly dynamic database environment that demands frequent approximate nearest-neighbor (ANN) operations [40], typically implemented through embedding vector indexes [23]. Such operations introduce a new and substantial source of overhead in agent serving when the index scale becomes large.

Existing agentic memory implementations largely focus on functional support while lacking performance-oriented optimization. As shown in Figure 1, for popular memory-based workflows [53,73], the memory operational cost grows sharply with memory size, reaching more than 82% of the total execution time. Meanwhile, most existing vector database systems fall short in supporting agentic memory: they either optimize only a static index [9,17,26,72,81], or rely on batch-oriented updates [49,50,60,74] designed for periodic maintenance in traditional databases [63], making them ill-suited for highly dynamic and fine-grained memory operations.

In this work, we present Pancake, a multi-tier memory management system designed to address the key challenges of implementing an agentic memory system:

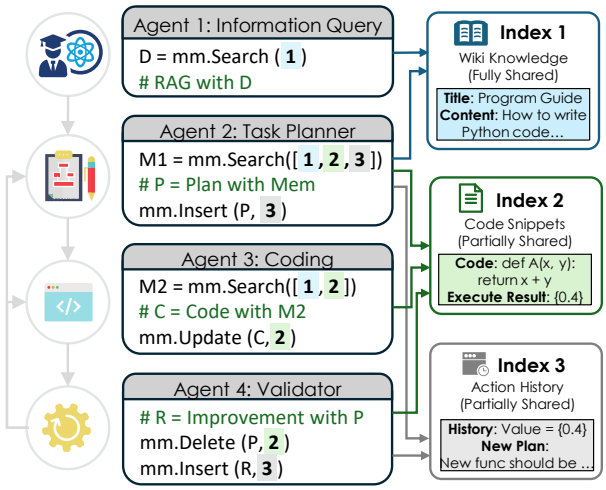

Figure 2: An example of multi-agent memory in Pancake.

First, for a **single agent**, a key limitation of existing vector search systems is their inability to efficiently handle the frequent, fine-grained updates characteristic of memory workloads [53, 73]. Existing incremental indexing methods [49, 74] are designed for large-batch insertions in traditional databases [34, 63] and rely on direct in-place inserts with periodic rebalancing. Under the small-batch insertion patterns and interleaved search of agent workloads, inserted vectors are often scattered across various clusters due to high-dimensional distance concentration [20] despite strong semantic coherence, degrading both search efficiency and recall.

To address this inefficiency, we explicitly incorporate agent-specific access behaviors into index construction and maintenance. Specifically, Pancake exploits both intra-agent and inter-request locality through a multi-level cache index that progressively promotes related vectors to upper levels, improving search ordering and enabling early termination. To guide caching behavior, Pancake models each agent's memory access pattern as finite-state machine (FSMs) with continuous updating and merging, enabling index cluster construction closely aligned with the agent's workload.

Second, supporting **multi-agent workloads** is challenging, as they frequently invoke different sets of agents to perform memory searches at runtime [54]. Using conventional two-level index structures [17] while maintaining separate indexes for each agent is inefficient. At the upper level, this design must traverse the coarse index of every agent, even when only a subset is involved in the query, leading to excessive coarse-search overhead. At the lower level, it uses uniform clusters for the shared memory part, ignoring the inconsistent access patterns across agents, thus causing misaligned cluster organization and inefficient fine-grained search.

Pancake addresses these challenges with a hybrid graph that connects multiple agents' indexes into a unified structure, enabling the upper-level coarse search to be performed through a single graph traversal. Pancake also records agent-specific access patterns for each cluster, to reduce cross-agent search overhead caused by inconsistent access patterns.

For programmability in multi-agent applications, Pancake provides a simple Python interface that supports operations over arbitrary memory scopes, including different shared and local memory parts, which existing frameworks do not offer. Figure 2 shows a multi-agent code-generation setup, where different agents flexibly operate over knowledge, code, and history memories with Pancake interface.

Third, modern LLM serving systems are typically deployed at **GPU-CPU platforms** [36, 86], which creates an opportunity to accelerate memory operations. Existing vector databases can only reside entirely on the GPU [32, 33], or GPU caching mechanism for static indexes [61, 84]. However, in agentic serving scenarios, the coexistence of large-scale memory bases [53] and LLM inference engine severely restricts available GPU memory. More importantly, the frequent memory updates makes static caching techniques infeasible to apply. To fully utilize resources, Pancake implements CPU–GPU coordinated index management to accelerate hotspot cluster computation, with an insertion buffer design and asynchronous transfers for low-latency online updates.

In summary, the contribution of this paper is as follows:

- We introduce Pancake, the first multi-tier memory management system tailored for multi-agent applications. Pancake exploits agent workload characteristics to optimize update strategy for single-agent memory, cluster construction for multi-agent coordination, and dynamic CPU–GPU collaborative execution.
- Pancake can be directly integrated into agent workflows like Mem-GPT [53] and plugged into mainstream agentic frameworks like LangChain [1] and LlamaIndex [44]. It provides a concise API through which agents can perform memory operations across flexible memory scopes.
- Extensive experiments across diverse agent datasets show that Pancake delivers over 4.29× average end-to-end performance speedup compared with existing memory libraries, and reduces the memory-operation time share to an average of 3.2% under large-scale database.

2 Background and Related Work

2.1 Memory-based Agent and ANN

A memory-based agent typically perform three operations: LLM Generation; Memory Search, which retrieves items relevant to the current context; and Memory Update, which inserts, deletes, or modifies items in the memory store. As shown in Figure 3, different agent roles induce different operation patterns. For instance, multi-turn dialogue agents search and

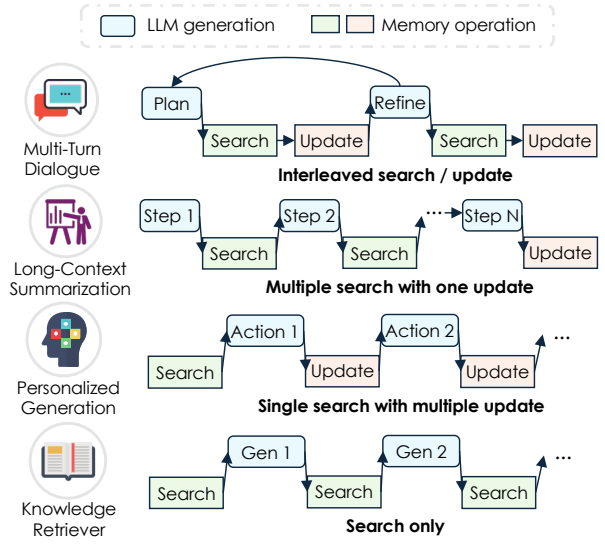

Figure 3: Memory-based agents and their workflows.

update memory at every step to remain consistent with the interaction history [53, 78], whereas context-summarization agents search and generate for several rounds and then insert a compressed memory item [10, 52, 79].

These search and update memory operations inherently introduce requirements for approximate nearest neighbor (ANN) queries. ANN is typically implemented through vector databases, where textual information is encoded into vector embeddings [14, 66] and relevance is quantized based on vector similarities [11]. Such ANN queries occur repeatedly throughout an agent's step-wise generation, their latency and accuracy therefore become increasingly critical to the overall performance of modern LLM serving systems.

To support memory operations, existing memory-based agents such as Mem-GPT [53] and A-Mem [73] provide their own memory implementations, and open-sourced agent frameworks like LlamaIndex [44] and LangChain [1] also offer built-in storage interfaces. However, these modules emphasize functionality and rely on suboptimal indexing and searching implementations. As the memory size grows, their query latency increases sharply, reaching more than 99% of the end-to-end runtime at scale. This trend highlights the need for a scalable and efficient agentic memory framework.

2.2 Dynamic Vector Database

Numerous vector-database frameworks [9, 17, 22, 26, 65] have explored techniques for efficient search by organizing vectors into structured storage formats, known as indexes. Among them, the Inverted File (IVF) index [27] is widely used: it partition vectors into clusters and rank these clusters by the distance between their centroids and the query. Only the top-*nprobe* clusters are selected for vector-wise search, making *nprobe* a tunable accuracy–efficiency trade-off [56]. We refer to cluster selection as *coarse search*, and to the search within the selected clusters as *fine search*. Coarse search typically relies on a Flat index or graph-based indexes such as HNSW [48] or Vamana [26] in large-scale settings.

However, existing frameworks are primarily designed for read-only scenarios like RAG, and therefore assume a static vector database with one-shot, full-index construction. Such designs are incompatible with agentic memory workloads, where frequent updates make reconstruction prohibitively expensive. To support online updates, several dynamic vector-database techniques have been proposed [49, 50, 60, 70, 74]. For example, SPFresh [74] avoids global rebuilding through in-place inserts and lightweight local rebalancing, while Quake [50] uses a hierarchical cluster structure and adaptively splits clusters based on access frequency. However, these designs typically assume periodic, batch-oriented updates consistent with traditional database workloads [35, 63]. In contrast, agentic memory serving involves highly frequent updates that interleave closely with search operations, leading to degraded efficiency and accuracy in such systems.

3 Motivation

3.1 Inefficient Update Strategy for Single-Agent Memory Access

In this section, we analyze single-agent memory access patterns and show that existing vector database maintenance algorithms struggle to efficiently handle frequent-update workloads. In typical agent-serving systems, an agent processes many independent requests, each involving multi-step LLM generation and frequent interleaved search–insert operations. Examples include coding agents receiving continuous user tasks [75] and scientific agents analyzing large batches of experimental data [80].

For vector insertion during memory updates, existing dynamic vector databases [49, 50, 74] typically adopt in-place inserts with periodic updates, as shown in Figure 4. New vectors are inserted directly into the nearest clusters, with distances calculated between the cluster centroids. Reconstruction is triggered only when the size or the semantic shift [49] of the cluster reaches a threshold. This strategy is effective for large-batch scenarios, while becomes suboptimal with interleaved small-batch search and insert operations.

Scattered Cluster Problem of In-Place Insertion. A key issue of in-place insertion is that new vectors inserted into a large pre-clustered index often get scattered across many clusters, even when they are semantically close. As shown in Figure 4(a), across 100 requests from several agent datasets [13, 15, 46], memory items from the same agent are dispersed into up to 175 clusters, with 38%–100% of these clusters being accessed with frequency less than 5%. This behavior stems from the high-dimensional shell effect [3, 6],

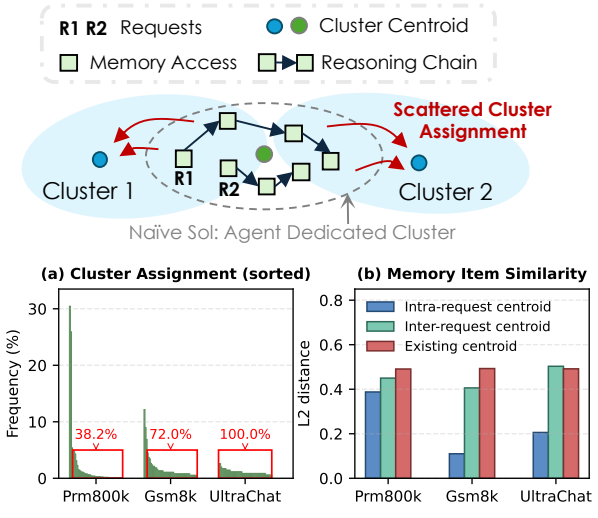

Figure 4: Direct in-place updates scatter the new vectors into a large number of existing clusters, leading to degradation in efficiency and recall. A naive solution is to leverage intra-agent locality and maintain dedicated clusters for a agent.

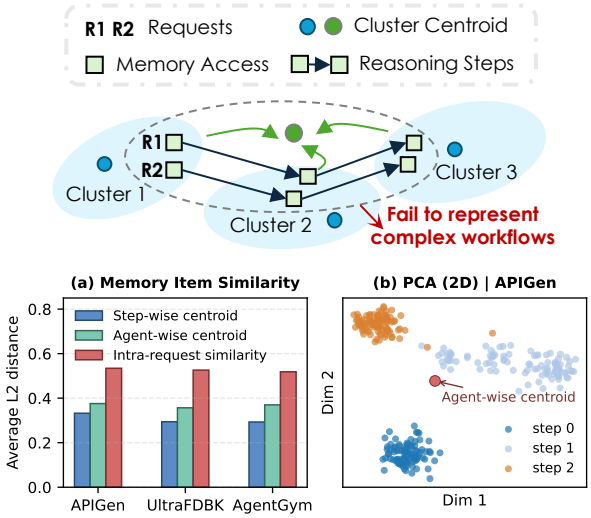

Figure 5: For more complex workloads, locality across multiple reasoning steps of different requests can be observed. This makes naive dedicated clusters for the agent inefficient, as it fails to capture step-wise clustering.

where points concentrate near the surface of a hypersphere, causing small semantic variations to translate into large differences in centroid distance calculations. Thus, even highly related memory items may be inserted to different clusters.

Such scattered cluster assignments bring challenges for both efficiency and accuracy. First, this forces scanning a larger number of clusters to retrieve semantically related items, incurring extra computation over mostly irrelevant vectors. Second, items in such scattered clusters become harder to locate by centroid distances, and their clusters may be eliminated during the coarse search stage, leading to drop in recall.

To tackle this problem, we first study two locality characteristics of agent memory, which provide critical guidance for designing effective clustering strategies.

Intra-Agent Locality. As shown in Figure 4(b), requests in agentic workflows insert memory items that remain highly coherent across steps and across requests of the same agent. The distances between each item and (i) the centroid of the memory items in the request and (ii) the aggregated centroid of all the agent's memory items are both substantially smaller than the distances to existing large clusters in the database. This phenomenon is particularly pronounced in task-focused workflows (e.g., mathematical reasoning [12]). A straightforward strategy is therefore to assign a dedicated cluster to each agent's insertion requests. However, this approach is insufficient in more complex workflows, where multi-step reasoning introduces cross-step transitions that cannot be captured by a single cluster (we will detail this in the next paragraph).

Inter-Request Step-wise Locality. Beyond intra-agent locality, more complex workflows reveal an additional layer of structure: memory items from the same reasoning step across different requests tend to cluster together. For example, in tool-augmented agents [57], the planning, tool-calling, and reflection steps across different requests tend to access similar regions of memory. As shown in Figure 5(a), memory items belonging to the same reasoning step across different requests demonstrate higher similarity, compared to the intra-request and intra-agent similarity. Figure 5(b) further illustrates the clustering patterns of memory items through 2-dimensional PCA visualization in the tool-calling dataset [46] with 100 requests, where three clusters emerge, each corresponding to an individual step of the workflow.

This step-wise organization induces frequent transitions across multiple clusters. Therefore, although maintaining a single dedicated cluster for each agent works for simple workflows in Figure 4, it is insufficient to capture the step-wise structures in more complicated scenarios, as shown in Figure 5. The green centroid becomes semantically unrepresentative, ultimately degrading accuracy and search efficiency.

In this work, we aim to optimize the agent memory management considering both intra-agent and step-wise locality. Compared to existing approaches [49,74], Pancake introduces more efficient cluster assignment and construction strategies that align with the agent's complex memory access patterns.

3.2 Challenges for Multi-Agent Memory Index Management

Beyond the memory inefficiency for a single agent, we identify the challenges of effectively managing and searching across multiple agent memories, which is a clear need for today's agentic workloads. In a typical multi-agent setting, each

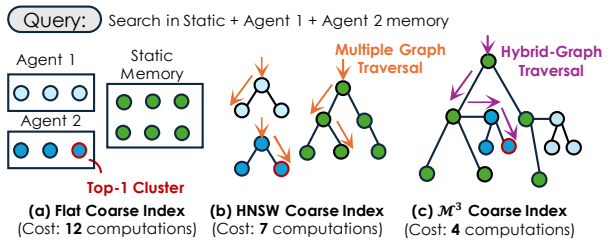

Figure 6: Coarse search costs with different index methods.

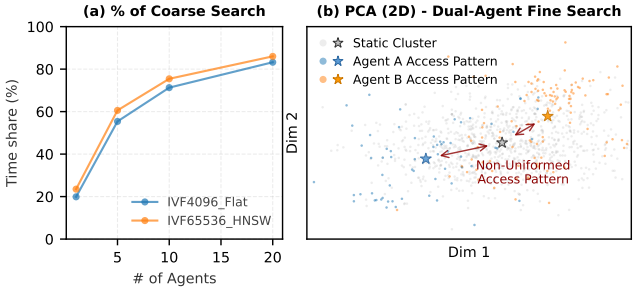

Figure 7: Coarse and Fine Search Challenges in Multi-Agent Memory. (a) Coarse search overhead grows rapidly as the number of agents increases. (b) When two agents access the same cluster in the static memory, their access patterns exhibit non-uniform distributions; circles denote accessed vectors, and stars denote the centroids formed by those vectors.

agent continuously updates its local memory, yet may search on the memories of other agents. For example, in generative-agent simulations like AI Town [54], each agent records its own action and observation histories, but relies on information originating from other agents' memories to plan behaviors and coordinate group activities. Because different agents become active or interact at different moments, the set of agent memories to search also varies over time.

This brings a clear demand for memory frameworks to support flexible specification of the memory search scope. However, existing ANN libraries [9, 17, 26] only provide interfaces for maintaining and querying on a single index for a given vector database, while offer no native support for search operations across different vector databases.

A straightforward approach is to maintain independent indexes for each agent's memory. When querying the memories of different scopes, the system searches the corresponding indexes and then merges the results. Although this approach can be implemented directly using existing library interfaces, it suffers from search efficiency issues, described as follows.

Excessive Coarse Search Cost. Large-scale vector indexes typically adopt a two-step search: a coarse search first selects the nearest clusters based on centroid distances, and a fine search is then performed within the selected clusters. When independent indexes are maintained for each agent, a wide-scope query must traverse the coarse index of every agent.

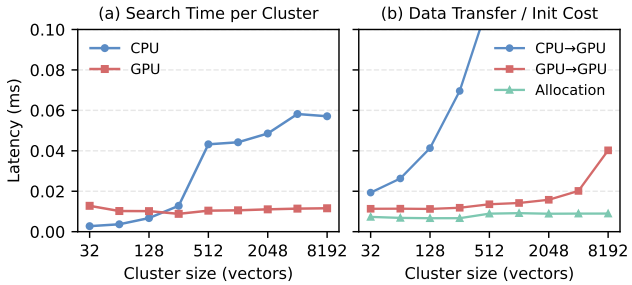

Figure 8: Comparison of operation costs on GPU and CPU, including (a) search time and (b) data transfer and allocation. The results are sampled on the MS MARCO [51] dataset.

As shown in Figure 6, when querying two agents and the static memory: (a) using a Flat index requires computing the distances to all centroids, and (b) using HNSW [48] requires a full traversal of the coarse-index graph of every agent.

We observe that such multi-index search patterns cause a significant amplification of coarse search cost when the number of agents increases. As shown in Figure 7(a), with the two commonly recommended Faiss indexes for large-scale settings [17], the cost of coarse-grained search rises sharply and exceeds 80% of the total latency when the number of agents reaches 20. Therefore, it becomes necessary to reorganize coarse indexes across agents, thereby reducing search costs during search across different agent memories.

Non-Uniform Fine Search Patterns across Agents. We further observe fine search inefficiency due to the different agent memory access patterns. As shown in Figure 7(b), when multiple agents query the same cluster in a memory index (constructed with static memory base [53]), the vectors they access differ markedly in distribution and clustering behavior. This divergence causes the effective centroid for each agent to shift in the embedding space. This divergence causes the centroids formed by each agent's accessed vectors within the same cluster to shift noticeably in the embedding space.

This finding indicates that the optimal fine-index organization is highly scope-dependent: for example, an index layout optimized for Agent 1's access pattern may be poorly aligned with Agent 2's pattern. As a result, Agent 1's cluster organization may force Agent 2 to compute over many irrelevant vectors and potentially suffer degraded recall. Such disalignment makes it necessary for coordinated organization of fine indexes across multiple agents.

To address the above issues, Pancake introduces a hybrid graph for efficient coarse search within only one graph traversal, as illustrated in Figure 6. Pancake also aligns fine-index access by associating each cluster with the pattern recognition of other agents, enabling optimized cross-agent search performance.

3.3 Difficulties for GPU-CPU Collaboration

In this section, we explore how to fully utilize the hardware resources of the GPU-CPU platform, which is widely adopted in LLM inference [36, 86]. Vector search involves high-dimensional floating-point computation and therefore benefits substantially from GPU acceleration. As shown in Figure 8(a), we characterize the performance advantages of CPU and GPU execution. When the number of vectors per cluster is small (< 256), CPU-based computation exhibits lower latency. In contrast, once the cluster size reaches a moderate range ($\geq$ 512), GPU-based vector search achieves a clear speedup of more than 3×. The GPU search latency remains largely stable as the cluster grows, since the dominant overhead arises from kernel launch rather than computation. Prior work has extensively explored fully GPU-resident indexes [32] and search frameworks [84].

However, large-scale vector databases (often over 100 GB [19]) place heavy demands on GPU memory and make it impractical for the GPU to store the entire index. The large model weights and KV cache further exacerbate this pressure. This necessitates an on-demand data transfer mechanism between the CPU and GPU. However, such transfer introduces significant overhead, typically far exceeding the actual computation time, as shown in Figure 8(b). To address this challenge, existing hybrid CPU–GPU designs employ hotspot caching and offloading [24, 33, 43, 61] for large-scale indexes.

The highly dynamic nature of agent memory introduces an additional dimension of complexity for maintaining consistency in CPU–GPU co-managed indexes. Hotspot clusters in agent memory are not only frequently queried but also frequently updated. However, because CUDA lacks efficient mechanisms for concurrent dynamic list expansion, clusters cached on the GPU cannot flexibly support frequent insertions. Prior work primarily supports cache management for static indexes, while performing updates on the CPU index and retransferring the modified clusters back to the GPU incurs prohibitive eviction and transfer costs.

To address the above challenge, Pancake implements a GPU–CPU coordinated dynamic index management scheme based on insertion buffers and asynchronous transfers. This design enables dynamically extensible hotspot clusters to be accelerated during both search and update operations.

4 Pancake: Methods and System Design

4.1 Overview

In this work, we present Pancake, a multi-tier ANN-based system designed to meet the demands of dynamic agentic memory workloads. Pancake follows a coordinated multi-tier design that consists of: (i) Multi-level, cache-inspired index orchestration informed by trajectory-based agent workload embeddings, enabling locality-aware search and update behavior; (ii) Multi-layer memory storage that supports efficient sharing, reuse, and migration of memory across agents; and (iii) Multi-device efficient execution with dynamic hotspot detection and cross-device consistency management to fully leverage heterogeneous CPU–GPU resources.

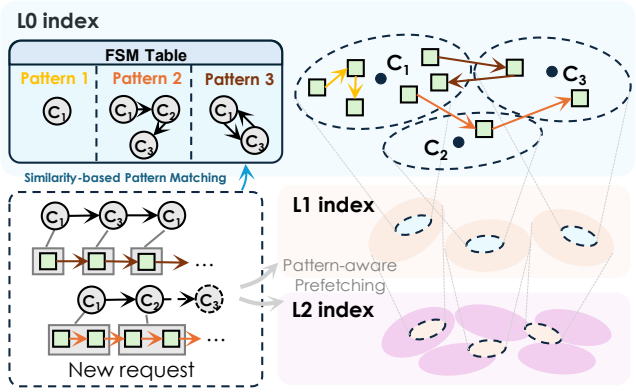

Figure 9: Three-level memory index cache to optimize search efficiency, with FSM-based modeling for access patterns.

4.2 Pattern-Driven Multi-Level Index Cache

Existing dynamic ANN methods either rely on streaming insertion and local rebalancing [49, 50, 74], or on coarse-grained buffering and periodic merging [60, 87]. Both approaches lack awareness of agent-level workload patterns, including intra-request locality and inter-request step-wise locality.

Three-Level Cluster Caching. We employ partial caching to resolve the mismatch between localized access operations and the coarse-grained cluster structure of the underlying ANN index. As shown in Figure 9, each upper level index forms a subset of the level below, but is organized to more closely reflect the agent's intrinsic memory-access patterns.

Search and update over the index always begin at the top level. L0 maintains a table S that tracks the most frequently accessed N_p tiny clusters, which contains the most recently accessed vectors to preserve the agent's temporal locality. When an L0 cluster overflows, evicted vectors are written back into the L1 index. L1 also maintains N_p intermediate clusters. It caches the top-k' neighbors for each search and update, where k' is slightly larger than the actual retrieval parameter k, allowing L1 to store the broader neighborhood around frequently accessed vectors. Finally, once an L1 cluster exceeds a predefined size threshold, it is merged with the L2 clusters, forming a stable and coarse-grained structure.

We leverage the early termination mechanism [4] in vector search to accelerate computation using cached data. During search, whenever all top-k candidates at the current level have distances smaller than $\alpha_{et} \cdot d_{agent}$, we skip computation at the next level. Here, d_{agent} denotes the average top-k distance across recent queries of the same agent. In practice, setting

$\alpha_{et} = 0.6 \sim 0.8$ provides a strong trade-off between efficiency and accuracy. We further introduce a verification mode in the system: after an early return, the system optionally performs the complete search in the background. This enables dynamic adjustment of α_{et} without incurring additional latency, while maintaining compatibility with LLM-coordinated speculative-generation workflows [24, 31, 83].

FSM-based Pattern Modeling. The agent memory access patterns can be modeled as a Finite-State-Machine (FSM):

$$P = (S, T), \qquad (c_i \to c_j) \in T,\ c_i, c_j \in S,$$

where S is the set of semantic cluster states. Each cluster $(c, \delta) \in S$ stores its cluster centroid c and the average intra-cluster vector deviation δ from the centroid. The transition set T captures the directed movement of memory accesses across cluster states. Such FSM abstraction preserves both semantic grouping and step-wise transition behavior.

The L0 index maintains a pattern table with N_p FSM entries. Given a new request with memory embedding sequence $(v_1, v_2, \ldots, v_t)$, we compute its similarity to pattern P_i based on prefix-state alignment and transition consistency:

$$\mathrm{sim}(P_i, v_{1:t}) = \sum_{k=1}^{t} I\big[(c_{k-1} \to c_k) \in T_i\big] \cdot \frac{\delta_k}{1 + |c_k - v_k|},$$

where $I[\cdot]$ is the indicator function. Using this similarity, each request identifies the best-matching pattern and infers the expected target cluster for subsequent memory accesses.

Pattern-based Reordering and Prefetching. Modeling the access pattern enables workload-aware search reordering. For each search operation, the cache manager matches its recent access sequence to a pattern in the FSM table and predicts the most probable L0 and L1 cluster. The search process then prioritizes the predicted cluster, enabling a more efficient search order and increasing the likelihood of early termination.

FSM-based modeling also enables prefetch-like behavior in the index cache. After each completed search or update, the system predicts the clusters likely to be accessed next. If these clusters have been evicted or written back, background prefetching is triggered to proactively refresh the cache ahead of time. Prefetching is carried out through an independent search, it can run in parallel with the agent's LLM-generation steps, creating additional opportunities to reduce overhead.

FSM Construction. Constructing such FSMs online is challenging, as agent memory accesses arrive in the form of embedding vectors rather than pre-labeled semantic clusters. Classical pattern-recognition approaches (e.g., PCA [2], HMMs [55]) are prohibitively expensive for high-dimensional and fine-grained online agent workloads. We therefore adopt a lightweight heuristic FSM construction and merging strategy.

When an agent request completes, the cache system first attempts to match it against an existing FSM in the table. If no match is found, a new FSM is created. During creation, each access in the request sequence becomes an independent state, and states are subsequently merged according to the maximum number of states N_S and the minimum merging distance d_{merge}. If the number of FSM entries exceeds N_p, the system merges two FSMs with the highest similarity, producing a compact and continuously updated FSM table.

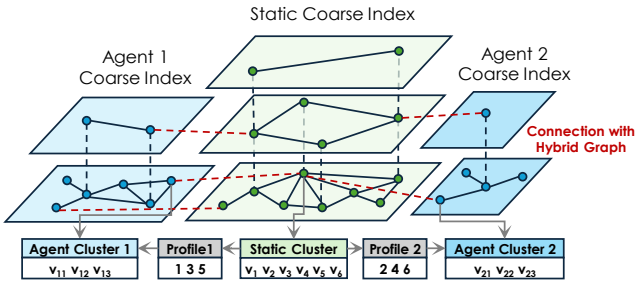

Figure 10: Multi-agent index management with hybrid graph and agent-specific pattern profiling on shared clusters.

4.3 Multi-Agent Indexing with Hybrid Graph

We propose a multi-agent–friendly index mechanism that incorporates coordinated coarse search and alignment. As shown in Figure 10, our design unifies the multiple coarse indexes into a hybrid graph structure, enabling efficient coarse search through graph traversal. Meanwhile, by associating each cluster with agent-specific memory access patterns, referred to as agent profiles, we further reduce overhead and improve recall for the cross-index operations.

Hybrid Graph Construction. We introduce a graph structure that connects the static memory and each agent's local memory. For the fine index, vectors in each static and agent local memory are stored only once, eliminating redundant storage. For the coarse index, each memory scope maintains its own coarse index, organized as a multi-level graph structure similar to HNSW [48]. Each layer forms a bounded-degree graph with up to M neighbors per node. Queries perform a greedy descent through the upper layers, followed by a best-first search at the bottom layer using a frontier of size ef_{search} to approximate the nearest neighbors.

Among multiple coarse indexes, we further introduce *inter-graph connections* to enable navigation across different memory scopes. Specifically, when maintaining each agent's coarse index, each node in the graph is additionally connected into the static coarse index with probability of $1/ef_{connect}$, thereby creating a controlled number of cross-agent portal nodes that support collaborative multi-agent search.

A cross-scope memory operation begins in the static coarse index entry and performs a BFS-like traversal. When the traversal encounters a node with an inter-connection to the target scope, the search adds the corresponding graph to the search frontier, and set the inter-connected node as the entry.

This enables seamless transition across memory scopes while avoiding unnecessary searching over irrelevant regions.

To determine a suitable value for ef_{connect}, we compare the density of the static coarse index with that of each agent-specific index. We measure the average centroid spacing within the private index (d_{agent}) and the static index (d_{static}), and set the inter-connection probability as

$$ef_{\text{connect}} = \min\left(\alpha_{ic} \cdot \frac{d_{\text{static}}}{d_{\text{agent}}},\ 1\right).$$

The intuition is as follows: when the static index covers a broader space, only sparse connections are needed; when the two spaces have similar density, denser connections help avoid cross-graph local minima. Empirically, we choose α_{ic} between 4 and 8 to balance efficiency and recall.

Search Optimization with Agent Profile. Due to highly non-uniform access patterns, clusters in the static memory exhibit different usage patterns across agents. However, the static memory index cannot adapt to these differences, leading to unnecessary search overhead and preventing the index from aligning with agent-specific access patterns.

To address this, Pancakeintroduces an *agent profile* mechanism for each static cluster. Specifically, every static cluster is associated with an agent-specific table that records the local IDs of recently accessed vectors within that cluster. Such list is maintained as a fixed-size sorted list. Whenever the top-k results of a query fall inside the current cluster, the corresponding vector IDs are promoted to the front of the list. For subsequent accesses, when the agent revisits the cluster, it first retrieves the vectors referenced in its profile by their stored local IDs, enabling a better search order and increasing the likelihood of early termination. Because the profile maintains only vector IDs and a lightweight list structure, the additional storage and management overhead is negligible comparing to the high-dimensional embedding computations.

4.4 Dynamic GPU-CPU Index Coordination

To further leverage heterogeneous hardware resources, we introduce a GPU–CPU coordinated dynamic index management mechanism, as illustrated in Figure 11. Our heterogeneous design consists of a CPU-side insertion buffer and a GPU-side manager for hotspot-aware caching, onloaded search, and consistent cluster maintenance. Such a system enables memory-efficient hotspot acceleration and dynamic cluster organization across devices. This is particularly critical in the co-located serving scenario with LLMs [24], where the inference engine occupys tens of gigabytes of GPU memory.

Hotspot-aware Caching. In our hybrid index manager, GPU memory dynamically caches hotspot clusters to accelerate critical computation. For each CPU-resident cluster, the system tracks its access frequency and selects the most frequently accessed clusters according to a predefined GPU

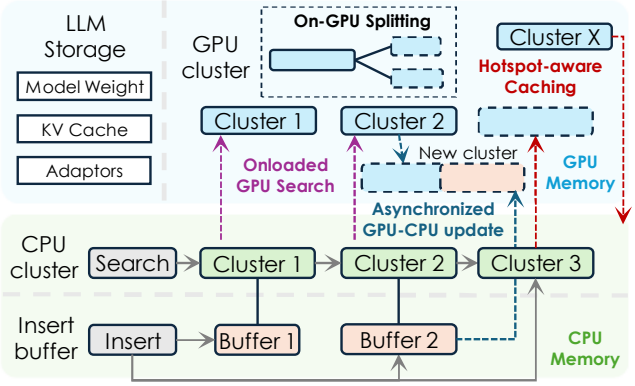

Figure 11: GPU-CPU coordinated index management to enable hotspot cluster computation acceleration.

memory budget. Whenever the hotspot set changes, the system performs cluster eviction and reallocation to keep the GPU cache aligned with the current workload. Data migration is through asynchronous CPU–GPU transfers to avoid high latency. However, insertions in the agentic memory may cause frequent staleness of the cached clusters.

CPU Insertion Buffer. As shown in the sampling results of § 3.3, the CPU computation time of a small set of vectors is lower than the GPU's cluster processing latency. Therefore, we maintain a per-cluster insertion buffer: once a cluster is resident on the GPU, subsequent insertions targeting that cluster are first accumulated in its CPU-side buffer of size B_{insert}. For all the searches targeting that cluster, computation is performed collaboratively using both the GPU-cached portion of the cluster and the vectors newly inserted the CPU buffer. The partial results from the two devices are then merged to produce the final results. Because the additional CPU-side search runs in parallel with the GPU computation and contributes only a small fraction of the overall processing time, the end-to-end request latency effectively matches that of a single-GPU execution. Based on this observation, we set B_{insert} to the largest cluster size where CPU-side search cost is lower than GPU-side search, which is 128 on our platform.

Asynchronized Consistency Management. When the insertion buffer becomes full, the corresponding GPU-cached cluster is resized, and the buffered vectors are migrated from the CPU to the GPU. To avoid the substantial latency caused by on-demand data transfers, we adopt a fully asynchronous cluster-expansion mechanism. The GPU-side index manager proactively allocates new space for clusters to expand and performs data transfers in parallel with online serving. Already cached data are migrated using low-cost GPU–GPU copies, while newly inserted buffer data are transferred through GPU–CPU copies. Once the new data transfer completes, the old GPU cluster is released, enabling seamless online cluster switching. This design eliminates both the waiting overhead associated with synchronous data movement and

the memory waste incurred by over-allocating GPU space.

On-GPU Cluster Splitting. The GPU cache introduces another optimization opportunity: accelerating the computation for cluster splitting. Clustering algorithms like K-means-based methods [16, 85] typically incurs a vector similarity cost that is multiple times higher than that of regular search. Thus, we implement a lightweight kernel based on GPU-based K-means algorithms [7, 41] to onload cluster splitting, avoiding the high computational load and latency on the CPU. Importantly, due to the locality of memory access, clusters that require splitting are usually cached on the GPU, so this technique can reduce the majority of splitting overhead.

5 Implementation

User Interface. Pancake provides a user-friendly Python interface that exposes simple primitives for agent-memory operations, including search, insert, update, and delete, with explicit specification of the target memory scope. Our initialization interface also supports loading from existing indexes, such as Faiss [17], enabling reconstruction that is friendly to IVF-based indexes. Operations submitted through the interface are batched, and adjacent operations of the same type are further grouped into a single batch to improve resource utilization.

Multi-threaded Index Construction. In Pancake, clusters are implemented as multithread-shared data structures, protected by shared-read and exclusive-write locks. Each cluster is associated with metadata, including its index identifier, multi-agent profiles, and its residency status across the multi-level cache and the GPU cache. We maintain a multithreaded execution pool that includes dedicated search threads, update threads, cache-management threads, and GPU-management threads. Insert and delete operations are also handled within the search threads, where items are updated based on the search results. Pancake adopts asynchronous invocation to ensure concurrency with LLM calls and to maintain compatibility with existing RAG-style systems [24, 28, 31].

6 Evaluation

6.1 Experimental Setup

Hardware. We conduct all experiments on a CPU–GPU hybrid server. Each node is equipped with one 64-core AMD EPYC 9534 processor and eight NVIDIA H100 GPUs with 80 GB of memory. The main control, scheduling, and computation logic of Pancake run on the CPU, while the LLM generation and GPU caching are performed on the H100 GPUs.

Baseline. For agent serving, we compare four memory design algorithms and their system implementations. These systems provide default ANN-based interfaces for memory management and retrieval. We evaluate them end-to-end by integrating their memory backends with LLM generation workloads. The baselines include: *A-Mem* [73]: backend for semantically evolving memory in long-term conversational retrieval. *MemGPT* [53]: backend for OS-style memory that swaps information between main context and external storage. *LlamaIndex* [44], vector-store backend in the RAG-oriented framework. *LangMem*: vector-store backend in the agentic framework LangChain [1].

For evaluating the performance of standalone vector databases, we compare our system against state-of-the-art dynamically updatable vector-index libraries. We use the vectors generated from the memory operations in our end-to-end agent workloads as input to these systems. The baselines include: *Quake* [50], structured and insert-friendly index library with dynamic hot-region–aware optimization. *SpFresh* [74], a large-scale vector search framework based on streaming insertion and localized, balancing-aware reconstruction. *DiskANN* [26, 60], open ANN library that combines an upper-layer graph with a lower-layer cluster index.

For ablation study, we also implement two dynamic maintenance strategies within our framework, including: *Pancake-IVF-Static*, which initializes an IVF index once and simply appends new vectors to the nearest centroid without any further maintenance. *Pancake-IVF-Split*, which performs cluster splitting when the size reaches a threshold, consistent with streaming-update and lazy-reconstruction strategies [49].

Dataset. We evaluate our system across diverse forms of agent dataset, including multi-turn human–agent dialogue datasets (UltraChat [15], UltraFeedback [13]), long chain-of-thought mathematical reasoning (Prm800k [42], Gsm8k [12]), and task-oriented agent datasets covering function calling (APIGen [46]) and environment interaction (AgentGym [69]).

Workload. We evaluate several representative memory access patterns, which can be observed in different types of agents:

–*One-Search-One-Insert*: Each generation step search the memory and updates it with the new output, typical for multi-turn conversational agents [53, 78].

–*Step-Search-Then-Insert*: Each step search the memory, but updates occur only at the end, typical for summarization or long-context compression agents [10, 52, 79].

–*Search-Then-Step-Insert*: Only the first step performs memory search, while the update occurs in each step, typical for personalized agents driven by user profiles [21, 67].

–*Search-Only*: The agent only queries memory without updates, typical for RAG-style agents [5, 8, 71].

Static Knowledge Database. We initialize the vector database similar to the Mem-GPT [53] setup, using the MS MARCO corpus [51], 8M passages in total, as the initial knowledge base. All embeddings are encoded using the E5 model [66] with 1024 dimension.

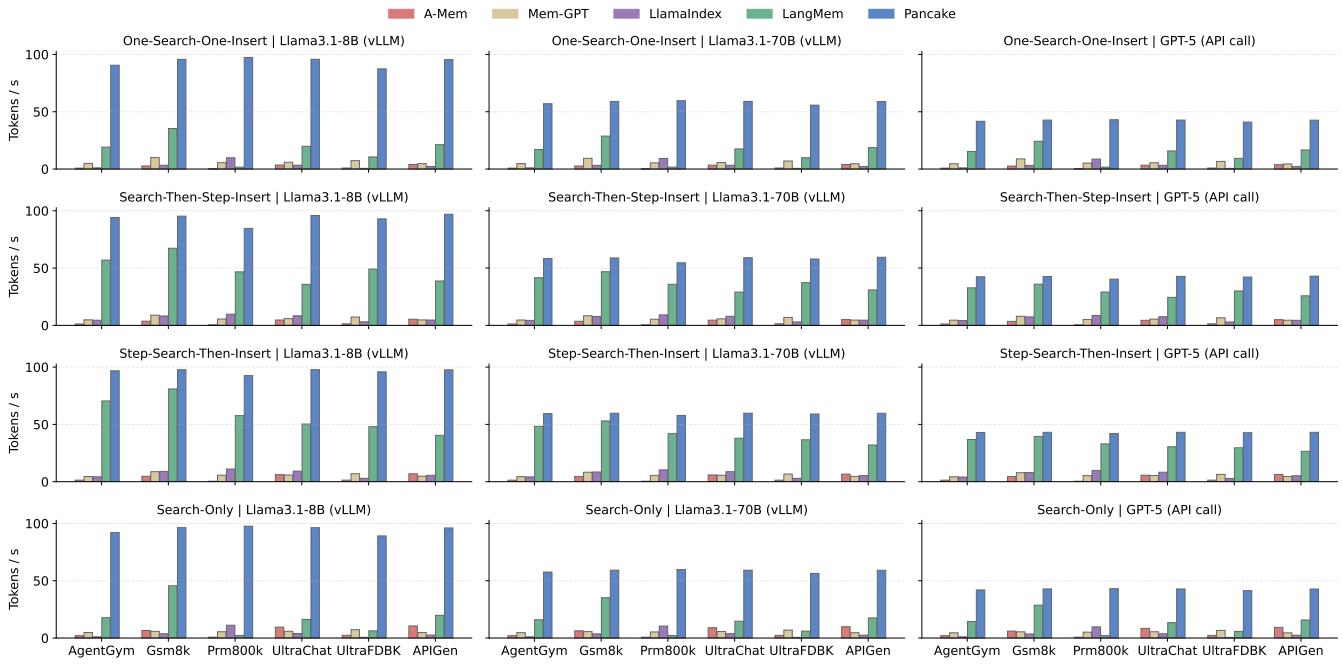

Figure 12: End-to-end throughput comparison between Pancake and other agentic frameworks, in a single agent scenario across four different access patterns. The experiments are conducted with vLLM [36] for Llama models [62] and API calls for GPT-5.

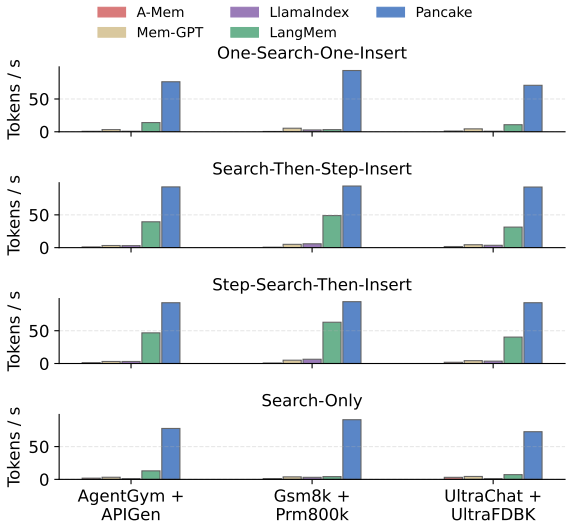

Figure 13: End-to-end throughput comparison in two-agent mixed workload, conducted with Llama3.1-8B.

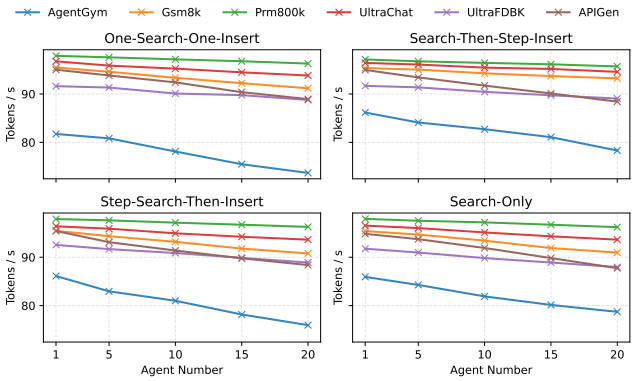

Figure 14: Scaling behavior of end-to-end throughput with an increasing agent number. The experiments are conducted with Llama3.1-8B.

6.2 Overall Performance

In this section, we evaluate the end-to-end improvements on memory-based agents performance when using Pancake.

Single-Agent Throughput. We compare different memory management libraries in single-agent settings across multiple models and datasets, as shown in Figure 12. Across both local inference servers [36] and remote API execution, Pancake consistently sustains stable single-agent request throughput, achieving end-to-end performance improvements ranging from 1.12× to 26.18×. On average, the speedup over existing libraries is more than 4.29×.

For the memory operations only, Pancake achieves speedups of more than 6.81×. The average memory operation time of Pancake accounts for less than 17.9%, and on average 3.2% of the total execution time. This demonstrates the effectiveness of Pancake in mitigating memory-related bottlenecks.

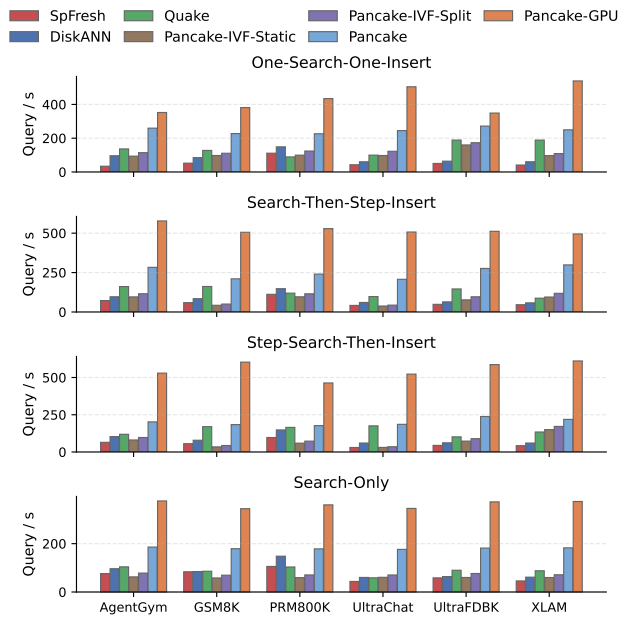

Figure 15: Query throughput of Pancake and existing vector database implementations, with the batch size set as 8.

In addition, we observe that workloads dominated by search operations, including One-Search-One-Insert and Search-only, exert a pronounced performance impact on baseline systems. This is because existing systems rely on suboptimal index constructions and maintenance strategies, which allow low-cost insertions but incur excessive search overhead.

Mixed-Workload Throughput. We evaluate the impact of two-agent mixed workloads on end-to-end performance, where each agent performs inserts on its private memory and searches on both shared and private memory. As shown in Figure 13, existing memory frameworks exhibit additional performance degradation, dropping by 29.9%∼55.9%. This degradation arises from separate memory instance maintenance and the lack of coordinated management across shared and private memory regions, which leads to interference between agents and amplifies operation overhead. In contrast, Pancake leverages its hybrid-graph design to enable efficient cross-agent search and index alignment, thereby preserving search locality and reducing redundant scans, Pancake limits the performance drop to no more than 9.8% under mixed workloads.

Multi-Agent Scalability. We construct varying numbers of memory-based agents and execute distinct requests over the same dataset, then measure overall throughput with operations across shared and private memory regions. As shown in Figure 14, Pancake achieves near-linear scalability in multi-agent settings. With up to 20 concurrent agents (the typical scale of common multi-agent frameworks [39, 68]), the end-to-end performance degradation remains below 10.2%.

We also observe that more complex dataset sequences, such as AgentGym and APIGen, exhibit larger performance drops. This is because broader dataset coverage increases the number of nodes traversed during the coarse-level graph search, resulting in proportionally higher search overhead.

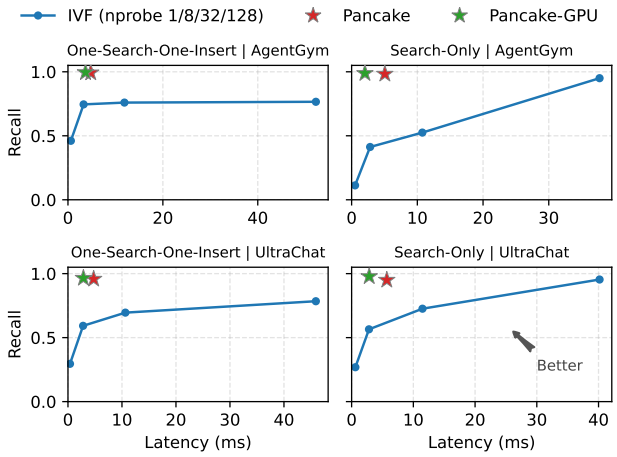

Figure 16: Tradeoff between recall and query latency over different indexing strategies.

6.3 Comparison with Existing Vector Database

Query Throughput Improvement. We compare Pancake with existing vector databases and indexing strategies under online serving workloads. As shown in Figure 15, Pancake consistently improves throughput across memory-intensive agent-serving scenarios, achieving 1.9× to 4.2× average speedups over the baselines. These speedups stem from cache and index designs tailored to agent memory-access patterns, which is overlooked in prior work. When leveraging GPU acceleration, Pancake further achieves an additional 2.2× performance gain, resulting in more than 3.9× speedup over other baselines. This demonstrates the effectiveness of dynamically coordinating GPU resources through our management mechanisms.

Tradeoff between Efficiency and Recall. We compare recall–latency trade-offs under both mixed search–update and search-only workloads. As shown in Figure 16, directly applying IVF index yields relatively low recall: in the search-only setting, IVF must scan up to 128 clusters to reach recall above 0.9. Under the search–update workload, IVF suffers an even larger recall drop because newly inserted vectors are scattered across different clusters, leading to reduced locality and suboptimal index organization.

By exploiting agent locality and memory-access patterns, Pancake effectively leverages its caching mechanism to reduce latency while maintaining high recall. Moreover, with coordinated GPU processing, Pancake can scan additional clusters while simultaneously serving cache hits, achieving lower latency together with a slight improvement in recall.

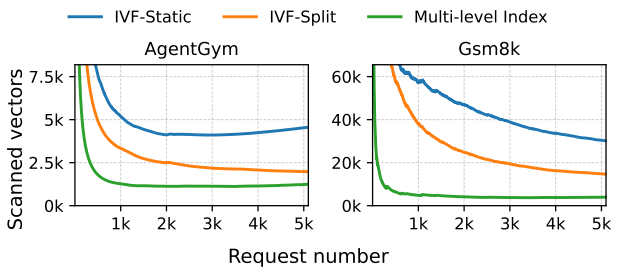

Figure 17: Number of scanned vectors to achieve fully recall of top-5 memory items. The insertion-to-search ratio is 1:1.

6.4 Ablation Study

In this section, we conduct detailed comparative experiments on the optimization techniques and provide an in-depth analysis of their effects and the root causes of the improvements.

Optimized Index with Multi-level Cache. We compare how different dynamic maintenance strategies affect the search costs with dynamism. A lower number of scanned vectors indicates that the index has evolved into a structure better aligned with the current agent's access pattern and provides stronger early-termination opportunities, thereby improving performance. As shown in Figure 17, performing IVF-Static updates leads to significantly higher scan counts, because newly inserted memory items are distributed across many clusters rather than being localized. IVF-Split eventually reduces the number of scanned vectors with sufficient insertions and the stable clusters formed. However, the long pre-convergence phase can be observed since the index cannot rebalance until the splitting threshold is reached. In contrast, our multi-level index cache effiently exploits agent-specific spatial and temporal locality, allowing it to stabilize at a low scan cost much earlier. This early adaptation leads to up to 2.23× latency reduction over long-serving workloads.

Search Efficiency with Multi-Agent Index. We first compare the reduction in coarse index search cost under multi-agent index management. As shown in Figure 18(a), maintaining separate indexes for each agent leads to a near-linear increase in coarse search overhead as the number of agents grows. In contrast, our multi-index management employs a hybrid graph that interconnects agents' coarse indexes, enabling efficient navigation of the global search space and achieving more than a 20× reduction in coarse search cost.

We further compare the total search cost under different optimization strategies. As shown in Figure 18(b), the hybrid-graph construction reduces the number of vector similarity computations by up to 11.6% compared to independently constructed indexes. Moreover, when incorporating agent profiles, we can track each agent's access preferences within the static clusters, achieving an additional 21.8% reduction in average computation cost without modifying the global index layout. These results highlight the unique advantages

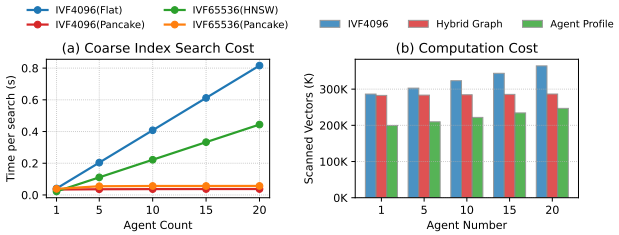

Figure 18: Efficiency improvements from multi-level index management on (a) coarse index search cost and (b) total computation cost.

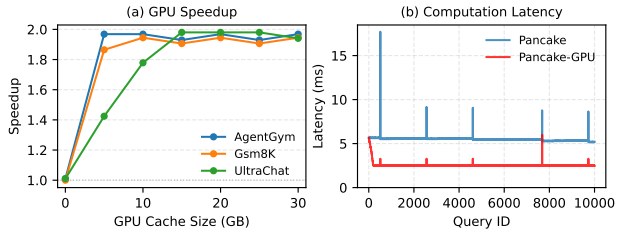

Figure 19: (a) GPU speedups under varying pre-allocated GPU cache sizes. (b) Computation latency of each query over the input workload with AgentGym dataset and 10GB GPU memory cache size. The insertion-to-search ratio is 1:1.

of Pancake in multi-agent index management.

Speedups with GPU Caching. We evaluate how GPU cache size affects performance. As shown in Figure 19(a), the GPU-accelerated version achieves up to 1.92× speedup over the CPU baseline and reaches a performance plateau with only 5∼15 GB of GPU memory. The effectiveness of GPU acceleration depends on the workload: conversational datasets distribute query-relevant clusters more widely, requiring a larger GPU cache to fully exploit acceleration.

We also examine latency over time under a mixed search–insert workload. As shown in Figure 19(b), the GPU version warms up quickly by caching hot clusters and maintains low, stable latency. Occasional spikes arise when streaming insertions trigger cluster splits, temporarily introducing additional computation. In our GPU-enabled design, most cluster operations are performed on the GPU, reducing the cost of these split events and keeping their impact minimal.

7 Conclusion

We presented Pancake, a multi-tier memory management system that bridges the gap between dynamic agentic memory and ANN-based vector indexing. Pancake leverages semantic locality for single-agent workloads, hybrid indexing for multi-agent memory management, and CPU–GPU collaborative indexing for acceleration. With a simple Python interface and support for flexible multi-scope memory operations, Pancake

integrates easily into existing agent frameworks. Experiments across diverse agent datasets show that Pancake significantly reduces memory operation overhead and delivers more than 4.29× average end-to-end speedup over existing implementations.

References

[1] langChain. https://github.com/langchain-ai/langchain, 2022.

[2] Hervé Abdi and Lynne J Williams. Principal component analysis. *Wiley interdisciplinary reviews: computational statistics*, 2(4):433–459, 2010.

[3] Charu C Aggarwal, Alexander Hinneburg, and Daniel A Keim. On the surprising behavior of distance metrics in high dimensional space. In *International conference on database theory*, pages 420–434. Springer, 2001.

[4] Vo Ngoc Anh, Owen de Kretser, and Alistair Moffat. Vector-space ranking with effective early termination. In *Proceedings of the 24th annual international ACM SIGIR conference on Research and development in information retrieval*, pages 35–42, 2001.

[5] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection. 2024.

[6] Kevin Beyer, Jonathan Goldstein, Raghu Ramakrishnan, and Uri Shaft. When is "nearest neighbor" meaningful? In *International conference on database theory*, pages 217–235. Springer, 1999.

[7] Janki Bhimani, Miriam Leeser, and Ningfang Mi. Accelerating k-means clustering with parallel implementations and gpu computing. In *2015 IEEE high performance extreme computing conference (HPEC)*, pages 1–6. IEEE, 2015.

[8] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pages 2206–2240. PMLR, 2022.

[9] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. Spann: Highly-efficient billion-scale approximate nearest neighborhood search. *Advances in Neural Information Processing Systems*, 34:5199–5212, 2021.

[10] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

[11] Gobinda G Chowdhury. *Introduction to modern information retrieval*. Facet publishing, 2010.

[12] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[13] Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Bingxiang He, Wei Zhu, Yuan Ni, Guotong Xie, Ruobing Xie, Yankai Lin, et al. Ultrafeedback: Boosting language models with scaled ai feedback. *arXiv preprint arXiv:2310.01377*, 2023.

[14] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

[15] Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. Enhancing chat language models by scaling high-quality instructional conversations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3029–3051, 2023.

[16] Yufei Ding, Yue Zhao, Xipeng Shen, Madanlal Musuvathi, and Todd Mytkowicz. Yinyang k-means: A drop-in replacement of the classic k-means with consistent speedup. In *International conference on machine learning*, pages 579–587. PMLR, 2015.

[17] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. *IEEE Transactions on Big Data*, 2025.

[18] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[19] Wikimedia Foundation. Wikimedia downloads.

[20] Damien François, Vincent Wertz, and Michel Verleysen. The concentration of fractional distances. *IEEE Transactions on Knowledge and Data Engineering*, 19(7):873–886, 2007.

[21] Tao Ge, Xin Chan, Xiaoyang Wang, Dian Yu, Haitao Mi, and Dong Yu. Scaling synthetic data creation with 1,000,000,000 personas. *arXiv preprint arXiv:2406.20094*, 2024.

[22] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. Accelerating large-scale inference with anisotropic vector quantization. In *International Conference on Machine Learning*, pages 3887–3896. PMLR, 2020.

[23] Yikun Han, Chunjiang Liu, and Pengfei Wang. A comprehensive survey on vector database: Storage and retrieval technique, challenge. *arXiv preprint arXiv:2310.11703*, 2023.

[24] Zhengding Hu, Vibha Murthy, Zaifeng Pan, Wanlu Li, Xiaoyi Fang, Yufei Ding, and Yuke Wang. Hedrarag: Co-optimizing generation and retrieval for heterogeneous rag workflows. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, pages 623–638, 2025.

[25] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International conference on machine learning*, pages 9118–9147. PMLR, 2022.

[26] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. Diskann: Fast accurate billion-point nearest neighbor search on a single node. *Advances in neural information processing Systems*, 32, 2019.

[27] Herve Jegou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence*, 33(1):117–128, 2010.

[28] Wenqi Jiang, Shuai Zhang, Boran Han, Jie Wang, Bernie Wang, and Tim Kraska. Piperag: Fast retrieval-augmented generation via algorithm-system co-design. *arXiv preprint arXiv:2403.05676*, 2024.

[29] Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7969–7992, 2023.

[30] Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7969–7992, 2023.

[31] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Shufan Liu, Xuanzhe Liu, and Xin Jin. Ragcache: Efficient knowledge caching for retrieval-augmented generation. *ACM Transactions on Computer Systems*, 44(1):1–27, 2025.

[32] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[33] V Karthik, Saim Khan, Somesh Singh, Harsha Vardhan Simhadri, and Jyothi Vedurada. Bang: Billion-scale approximate nearest neighbour search using a single gpu. *IEEE Transactions on Big Data*, 2025.

[34] Ralph Kimball and Joe Caserta. *The data warehouse ETL toolkit*. John Wiley & Sons, 2004.

[35] Ralph Kimball and Margy Ross. *The data warehouse toolkit: The definitive guide to dimensional modeling*. John Wiley & Sons, 2013.

[36] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.

[37] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

[38] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

[39] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for" mind" exploration of large language model society. *Advances in Neural Information Processing Systems*, 36:51991–52008, 2023.

[40] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. Approximate nearest neighbor search on high dimensional data—experiments, analyses, and improvement. *IEEE Transactions on Knowledge and Data Engineering*, 32(8):1475–1488, 2019.

[41] You Li, Kaiyong Zhao, Xiaowen Chu, and Jiming Liu. Speeding up k-means algorithm by gpus. *Journal of Computer and System Sciences*, 79(2):216–229, 2013.

[42] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *The Twelfth International Conference on Learning Representations*, 2023.

[43] Chien-Yu Lin, Keisuke Kamahori, Yiyu Liu, Xiaoxiang Shi, Madhav Kashyap, Yile Gu, Rulin Shao, Zihao Ye, Kan Zhu, Stephanie Wang, et al. Telerag: Efficient retrieval-augmented generation inference with lookahead retrieval. *arXiv preprint arXiv:2502.20969*, 2025.

[44] Jerry Liu. LlamaIndex. https://github.com/jerryjliu/llama_index, 11 2022.

[45] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[46] Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Juntao Tan, Weiran Yao, Zhiwei Liu, Yihao Feng, Rithesh RN, et al. Apigen: Automated pipeline for generating verifiable and diverse function-calling datasets. *Advances in Neural Information Processing Systems*, 37:54463–54482, 2024.

[47] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

[48] Yu A Malkov and Dmitry A Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence*, 42(4):824–836, 2018.

[49] Jason Mohoney, Anil Pacaci, Shihabur Rahman Chowdhury, Umar Farooq Minhas, Jeffery Pound, Cedric Renggli, Nima Reyhani, Ihab F Ilyas, Theodoros Rekatsinas, and Shivaram Venkataraman. Incremental ivf index maintenance for streaming vector search. *arXiv preprint arXiv:2411.00970*, 2024.

[50] Jason Mohoney, Devesh Sarda, Mengze Tang, Shihabur Rahman Chowdhury, Anil Pacaci, Ihab F Ilyas, Theodoros Rekatsinas, and Shivaram Venkataraman. Quake: Adaptive indexing for vector search. *arXiv preprint arXiv:2506.03437*, 2025.

[51] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. Ms marco: A human-generated machine reading comprehension dataset. 2016.

[52] Siru Ouyang, Jun Yan, I Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T Le, Samira Daruki, Xiangru Tang, et al. Reasoningbank: Scaling agent self-evolving with reasoning memory. *arXiv preprint arXiv:2509.25140*, 2025.

[53] Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. Memgpt: Towards llms as operating systems. 2023.

[54] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023.

[55] Lawrence R Rabiner. A tutorial on hidden markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 2002.

[56] Siddhant Ray, Rui Pan, Zhuohan Gu, Kuntai Du, Shaoting Feng, Ganesh Ananthanarayanan, Ravi Netravali, and Junchen Jiang. Metis: Fast quality-aware rag systems with configuration adaptation. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, pages 606–622, 2025.

[57] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.

[58] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023.

[59] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023.

[60] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. Freshdiskann: A fast and accurate graph-based ann index for streaming similarity search. *arXiv preprint arXiv:2105.09613*, 2021.

[61] Bing Tian, Haikun Liu, Yuhang Tang, Shihai Xiao, Zhuohui Duan, Xiaofei Liao, Hai Jin, Xuecang Zhang, Junhua Zhu, and Yu Zhang. Towards high-throughput and low-latency billion-scale vector search via {CPU/GPU}

collaborative filtering and re-ranking. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 171–185, 2025.

[62] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[63] Denny Vrandečić and Markus Krötzsch. Wikidata: a free collaborative knowledgebase. *Communications of the ACM*, 57(10):78–85, 2014.

[64] Hanchen Wang, Tianfan Fu, Yuanqi Du, Wenhao Gao, Kexin Huang, Ziming Liu, Payal Chandak, Shengchao Liu, Peter Van Katwyk, Andreea Deac, et al. Scientific discovery in the age of artificial intelligence. *Nature*, 620(7972):47–60, 2023.

[65] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 international conference on management of data*, pages 2614–2627, 2021.

[66] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.

[67] Zhen Wang, Yufan Zhou, Zhongyan Luo, Lyumanshan Ye, Adam Wood, Man Yao, and Luoshang Pan. Deep-persona: A generative engine for scaling deep synthetic personas. *arXiv preprint arXiv:2511.07338*, 2025.

[68] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024.

[69] Zhiheng Xi, Yiwen Ding, Wenxiang Chen, Boyang Hong, Honglin Guo, Junzhe Wang, Dingwen Yang, Chenyang Liao, Xin Guo, Wei He, et al. Agentgym: Evolving large language model-based agents across diverse environments. *arXiv preprint arXiv:2406.04151*, 2024.

[70] Wentao Xiao, Yueyang Zhan, Rui Xi, Mengshu Hou, and Jianming Liao. Enhancing hnsw index for real-time updates: Addressing unreachable points and performance degradation. *arXiv preprint arXiv:2407.07871*, 2024.

[71] Fangyuan Xu, Weijia Shi, and Eunsol Choi. Recomp: Improving retrieval-augmented lms with context compression and selective augmentation. In *The Twelfth International Conference on Learning Representations*, 2024.

[72] Qian Xu, Juan Yang, Feng Zhang, Junda Pan, Kang Chen, Youren Shen, Amelie Chi Zhou, and Xiaoyong Du. Tribase: A vector data query engine for reliable and lossless pruning compression using triangle inequalities. *Proceedings of the ACM on Management of Data*, 3(1):1–28, 2025.

[73] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025.

[74] Yuming Xu, Hengyu Liang, Jin Li, Shuotao Xu, Qi Chen, Qianxi Zhang, Cheng Li, Ziyue Yang, Fan Yang, Yuqing Yang, et al. Spfresh: Incremental in-place update for billion-scale vector search. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 545–561, 2023.

[75] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.

[76] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.

[77] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.

[78] Hongli Yu, Tinghong Chen, Jiangtao Feng, Jiangjie Chen, Weinan Dai, Qiying Yu, Ya-Qin Zhang, Wei-Ying Ma, Jingjing Liu, Mingxuan Wang, et al. Memagent: Reshaping long-context llm with multi-conv rl-based memory agent. *arXiv preprint arXiv:2507.02259*, 2025.

[79] Guibin Zhang, Muxin Fu, Guancheng Wan, Miao Yu, Kun Wang, and Shuicheng Yan. G-memory: Tracing hierarchical memory for multi-agent systems. *arXiv preprint arXiv:2506.07398*, 2025.

[80] Huan Zhang, Yu Song, Ziyu Hou, Santiago Miret, and Bang Liu. Honeycomb: A flexible llm-based agent system for materials science. *arXiv preprint arXiv:2409.00135*, 2024.

[81] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, et al. {VBASE}: Unifying online vector similarity search and relational queries via relaxed monotonicity. In *17th USENIX Symposium on*

Operating Systems Design and Implementation (OSDI 23), pages 377–395, 2023.

[82] Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents, 2024. *URL https://arxiv.org/abs/2404.13501*.

[83] Zhihao Zhang, Alan Zhu, Lijie Yang, Yihua Xu, Lanting Li, Phitchaya Mangpo Phothilimthana, and Zhihao Jia. Accelerating retrieval-augmented language model serving with speculation. *arXiv preprint arXiv:2401.14021*, 2024.

[84] Zili Zhang, Fangyue Liu, Gang Huang, Xuanzhe Liu, and Xin Jin. Fast vector query processing for large datasets beyond {GPU} memory with reordered pipelining. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 23–40, 2024.

[85] Weijie Zhao, Shulong Tan, and Ping Li. Song: Approximate nearest neighbor search on gpu. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*, pages 1033–1044. IEEE, 2020.

[86] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.

[87] Shurui Zhong, Dingheng Mo, and Siqiang Luo. Lsm-vec: A large-scale disk-based system for dynamic vector search. *arXiv preprint arXiv:2505.17152*, 2025.

[88] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19724–19731, 2024.